

BankPro Capstone Project

CI/CD Pipeline Deployment to AWS Cloud Using Jenkins, Docker, and Ansible

Prepared by: Ieng Lao

Course-End Project 1 — Application Deployment to AWS Cloud

Date: June 13, 2026

Table of Contents

1. Objective

The objective of this project is to demonstrate the end-to-end process of deploying a Spring Boot microservice application to the AWS cloud environment using a Jenkins pipeline, Ansible, and Docker. The project automates the CI/CD workflow, containerizes the application, and performs a seamless cloud deployment to an AWS EC2 instance, ensuring repeatability, scalability, and reduced manual intervention.

2. Problem Statement and Business Context

2.1 Scenario

BankPro, a leading financial services company, operates an AWS cloud infrastructure to deliver its banking and payment services to clients across the US and Europe. To enhance reliability and scalability of its core banking application, the company is adopting a containerized microservices architecture.

As the DevOps Engineer, the task is to automate the CI/CD pipeline for the core banking microservice using Jenkins, Ansible, and Docker — covering build, test, containerization, and deployment to AWS EC2, reducing downtime and improving service availability.

2.2 Industry Relevance

- Jenkins: Open-source automation server used to automate building, testing, and deployment.
- AWS Cloud (EC2): Provides scalable virtual machines used as the deployment target for the containerized application.
- GitHub: Version control platform serving as the source code repository for the Jenkins pipeline.
- Docker: Containerization platform used to package the application for consistent deployment.
- Ansible: Configuration management and deployment automation tool used to provision and deploy to the target EC2 instance.

3. Solution Architecture

The pipeline integrates source control, build automation, containerization, and configuration management into a single automated workflow:

```

GitHub (source repo)
  | (webhook on push)
  v
Jenkins Server (EC2 - controller)
  |-- Checkout source code
  |-- Build with Maven (compile, test, package JAR)
  |-- Build Docker image
  |-- Push image to Docker Hub
  |-- Run Ansible playbook
  v
Ansible (control node on Jenkins server)
  |-- Connect via SSH to App Server
  |-- Install Docker (if needed)
  
```

```
|-- Pull latest image from Docker Hub
|-- Stop old container / run new container
v
App Server (EC2)
Banking microservice container running on port 8989
Accessible via Swagger UI: /bank-api/swagger-ui/index.html
```

3.1 AWS Services and Components Used

Component	Purpose
Amazon EC2 (jenkins-server)	Hosts Jenkins, Docker engine, and Ansible control node
Amazon EC2 (app-server)	Target deployment server running the banking microservice container
VPC / Security Groups	Network isolation and controlled inbound access (SSH, Jenkins UI, app port)
Amazon EC2 Key Pairs	SSH key-based authentication for instance access and Ansible connectivity
Docker Hub	Container image registry for the banking application image
GitHub	Source code repository and CI trigger via webhook

4. AWS Infrastructure Setup

4.1 EC2 Instances

Instance	Purpose	AMI	Instance Type
jenkins-server	Jenkins + Docker + Ansible control node	Ubuntu 26.04 LTS	t2.medium
app-server	Target deployment server for banking app	Ubuntu 26.04 LTS	t2.micro

4.2 Security Group Rules

Instance	Port	Protocol	Source	Purpose
jenkins-server	22	TCP	Admin IP	SSH management
jenkins-server	8080	TCP	0.0.0.0/0	Jenkins UI and GitHub webhook
app-server	22	TCP	jenkins-server SG / private IP	SSH for Ansible
app-server	8989	TCP	0.0.0.0/0	Banking application / Swagger UI

Note: For production environments, inbound rules for ports 8080 and 8989 should be restricted to known IP ranges or placed behind a load balancer / VPN, rather than open to 0.0.0.0/0.

5. Jenkins Installation and Configuration

5.1 Install Java 21

Jenkins (current LTS) requires Java 21 or later. Java 17 was initially installed but Jenkins failed to start with the error "Running with Java 17 ... older than the minimum required version (Java 21)".

```
sudo apt install -y openjdk-21-jdk
sudo update-alternatives --set java /usr/lib/jvm/java-21-openjdk-amd64/bin/java
java -version
```

5.2 Install Jenkins

The Jenkins APT repository requires a GPG key in binary (dearmored) format. The key was imported via the Ubuntu keyserver to resolve a NO_PUBKEY signature error:

```
sudo apt install -y gnupg dirmngr
sudo gpg --no-default-keyring \
  --keyring /etc/apt/trusted.gpg.d/jenkins-keyring.gpg \
  --keyserver keyserver.ubuntu.com --recv-keys 7198F4B714ABFC68

echo "deb https://pkg.jenkins.io/debian-stable binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list

sudo apt update
sudo apt install -y jenkins
sudo systemctl enable jenkins
sudo systemctl start jenkins
```

Access Jenkins at <http://<jenkins-public-ip>:8080> and retrieve the initial admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

5.3 Install Required Plugins

Installed via Manage Jenkins -> Plugins -> Available plugins:

- Docker Pipeline
- Pipeline: Stage View
- SSH Agent
- Ansible
- GitHub Integration
- Maven Integration

5.4 Configure Tools

Manage Jenkins -> Tools:

- JDK: name = JDK21, JAVA_HOME = /usr/lib/jvm/java-21-openjdk-amd64
- Maven: name = Maven3, install automatically (version 3.9.x)

5.5 Install Docker on Jenkins Server

```
sudo apt install -y docker.io
sudo systemctl enable docker
```

```
sudo systemctl start docker
sudo usermod -aG docker jenkins
sudo usermod -aG docker ubuntu
sudo systemctl restart jenkins
```

5.6 Docker Hub Credentials

Added under Manage Jenkins -> Credentials -> Global credentials:

- Kind: Username with password
- Username: Docker Hub username
- Password: Docker Hub access token
- ID: dockerhub-creds

6. Ansible Setup and Connectivity

6.1 Install Ansible (Control Node)

```
sudo apt update
sudo apt install -y ansible
ansible --version
```

6.2 SSH Key-Based Authentication

Generated an SSH key pair for the jenkins user and copied the public key to the app-server's authorized_keys file:

```
sudo -u jenkins ssh-keygen -t rsa -b 4096 -f /var/lib/jenkins/.ssh/id_rsa -N ""
sudo cat /var/lib/jenkins/.ssh/id_rsa.pub

# On app-server:
echo "<jenkins-public-key>" >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
chmod 700 ~/.ssh
```

6.3 Inventory File

Created at /home/ubuntu/ansible/inventory.ini on the jenkins-server:

```
[appservers]
app-server ansible_host=172.31.20.121 ansible_user=ubuntu \
  ansible_ssh_private_key_file=/var/lib/jenkins/.ssh/id_rsa \
  ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

Permission note: /home/ubuntu was set to mode 750, which blocked the jenkins user from traversing the directory. Resolved with:

```
chmod 755 /home/ubuntu
```

6.4 Connectivity Verification

```
sudo -u jenkins ansible all -i /home/ubuntu/ansible/inventory.ini -m ping
# Result: app-server | SUCCESS => { "ping": "pong" }

sudo -u jenkins ansible all -i /home/ubuntu/ansible/inventory.ini -m shell -a "uptime"
# Result: app-server | CHANGED | rc=0 >> 19:08:33 up 37 min ...
```

7. Spring Boot Application and Repository Structure

A sample Spring Boot banking microservice was created with REST endpoints for Accounts and Customers, exposing a Swagger UI for API documentation.

7.1 Repository Structure

```
banking-app/
|-- pom.xml
|-- Dockerfile
|-- Jenkinsfile
|-- ansible/
|   |-- inventory.ini
|   |-- deploy.yml
|-- src/
|   |-- main/
|   |   |-- java/com/bankpro/
|   |   |   |-- BankingAppApplication.java
|   |   |   |-- controller/
|   |   |       |-- AccountController.java
|   |   |       |-- CustomerController.java
|   |   |-- resources/
|   |       |-- application.properties
|   |-- test/
|       |-- java/com/bankpro/
|           |-- BankingAppApplicationTests.java
```

7.2 application.properties

Configures the embedded server port and Swagger UI paths:

```
server.port=8989
server.servlet.context-path=/
spring.application.name=banking-app
springdoc.api-docs.path=/bank-api/v2/api-docs
springdoc.swagger-ui.path=/bank-api/swagger-ui.html
```

7.3 Dockerfile

A single-stage Dockerfile packages the pre-built JAR (built by Jenkins/Maven) into a lightweight JRE image:

```
FROM eclipse-temurin:17-jre-jammy
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8989
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Note: An earlier multi-stage Dockerfile that ran 'mvn clean package' inside the image failed with 'mvn: not found' because Maven is not present in the eclipse-temurin JDK image. Since Jenkins already builds the JAR with Maven, the Dockerfile only needs to package the resulting artifact.

8. Jenkins CI/CD Pipeline

8.1 Pipeline Stages Overview

Stage	Description
-------	-------------

Checkout	Clone source code from the GitHub repository (main branch)
Build with Maven	Compile and package the application as a JAR (mvn clean package)
Test	Run unit tests with Maven Surefire and publish JUnit results
Archive Artifact	Archive the built JAR as a Jenkins build artifact
Build Docker Image	Build and tag the Docker image with the build number and 'latest'
Push to Docker Hub	Authenticate and push both image tags to Docker Hub
Deploy via Ansible	Run the Ansible playbook to deploy the new image to app-server
Verify Deployment	Confirm the application responds at the Swagger UI endpoint

8.2 Jenkinsfile

```

pipeline {
    agent any
    environment {
        DOCKERHUB_CREDS = credentials('dockerhub-creds')
        IMAGE_NAME = "ienglao/bankingapp"
        IMAGE_TAG = "${BUILD_NUMBER}"
    }
    tools {
        maven 'Maven3'
        jdk 'JDK21'
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/iengster/banking-app.git'
            }
        }
        stage('Build with Maven') {
            steps { sh 'mvn clean package -DskipTests' }
        }
        stage('Test') {
            steps { sh 'mvn test' }
            post {
                always { junit 'target/surefire-reports/*.xml' }
            }
        }
        stage('Archive Artifact') {
            steps { archiveArtifacts artifacts: 'target/*.jar', fingerprint: true }
        }
        stage('Build Docker Image') {
            steps {
                sh "docker build -t ${IMAGE_NAME}:${IMAGE_TAG} ."
                sh "docker tag ${IMAGE_NAME}:${IMAGE_TAG} ${IMAGE_NAME}:latest"
            }
        }
        stage('Push to Docker Hub') {
            steps {
                sh "echo ${DOCKERHUB_CREDS_PSW} | docker login -u $
{DOCKERHUB_CREDS_USR} --password-stdin"
                sh "docker push ${IMAGE_NAME}:${IMAGE_TAG}"
                sh "docker push ${IMAGE_NAME}:latest"
            }
        }
        stage('Deploy via Ansible') {

```



```

        steps {
            sh """
            ansible-playbook -i /home/ubuntu/ansible/inventory.ini \\
            ansible/deploy.yml \\
            --extra-vars \"dockerhub_user=ienglao image_tag=${IMAGE_TAG}\"
            """
        }
    }
    stage('Verify Deployment') {
        steps {
            sh 'sleep 15'
            sh 'curl -fL http://172.31.20.121:8989/bank-api/swagger-ui/index.html'
        }
    }
} || exit 1'
}
}
post {
    success { echo "Deployment successful!" }
    failure { echo "Pipeline failed. Check logs." }
}
}

```

8.3 Ansible Deployment Playbook (ansible/deploy.yml)

```

---
- name: Deploy Banking App
  hosts: appservers
  become: yes
  tasks:
    - name: Install Docker
      apt:
        name: docker.io
        state: present
        update_cache: yes

    - name: Start Docker service
      service:
        name: docker
        state: started
        enabled: yes

    - name: Install pip3
      apt:
        name: python3-pip
        state: present

    - name: Install Python Docker module
      apt:
        name: python3-docker
        state: present

    - name: Pull latest image
      docker_image:
        name: "{{ dockerhub_user }}/bankingapp:{{ image_tag }}"
        source: pull

    - name: Remove existing container
      docker_container:
        name: bankingapp
        state: absent
      ignore_errors: yes

```

```
- name: Run new container
  docker_container:
    name: bankingapp
    image: "{{ dockerhub_user }}/bankingapp:{{ image_tag }}"
    state: started
    restart_policy: always
    ports:
      - "8989:8989"
```

Note: The original playbook used 'pip: name: docker' to install the Python Docker SDK, but failed with 'Unable to find any of pip3 to use'. This was resolved by installing python3-pip and python3-docker via the apt module instead.

9. GitHub Webhook Integration

A GitHub webhook was configured to automatically trigger the Jenkins pipeline on every push to the main branch, eliminating the need for manual builds.

9.1 Webhook Configuration

Setting	Value
Payload URL	http://<jenkins-public-ip>:8080/github-webhook/
Content type	application/json
Events	Just the push event
Active	Enabled

9.2 Jenkins Job Trigger

In the pipeline job configuration, under Build Triggers, 'GitHub hook trigger for GITScm polling' was enabled.

9.3 Troubleshooting Notes

- Initial webhook attempts failed because the Payload URL was malformed (duplicated 'http://' prefix and an extra slash before the port number, e.g. http://http://<ip>:8080/github-webhook/).
- After correcting the URL to http://<jenkins-ip>:8080/github-webhook/, GitHub successfully delivered the push event (visible under Recent Deliveries) and Jenkins automatically started a new build.
- Jenkins port 8080 needed to be open to 0.0.0.0/0 in the security group, since GitHub webhook delivery originates from GitHub's infrastructure IPs, not the user's IP.

10. Deployment Verification

10.1 Container Status on App Server

docker ps				
CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS
54609da5a77f	ienglao/bankingapp:6	"java -jar app.jar"	Up 4 minutes	
0.0.0.0:8989->8989/tcp				

10.2 Application Logs

```
docker logs bankingapp | grep "Tomcat started on port"
Tomcat started on port 8989 (http) with context path ''
```

10.3 Application Access

The Banking Application REST API, including Account Controller and Customer Controller endpoints, is accessible via Swagger UI:

```
http://<app-server-public-ip>:8989/bank-api/swagger-ui/index.html
```

11. Troubleshooting Summary

Issue	Root Cause	Resolution
apt update: NO_PUBKEY 7198F4B714ABFC68	Jenkins repo GPG key was ASCII-armored, not in expected binary keyring format	Imported the key in binary form via Ubuntu keyserver into /etc/apt/trusted.gpg.d/
Jenkins service failed to start	Jenkins LTS requires Java 21; Java 17 was the active default	Installed Java 21 and set it as the default with update-alternatives
systemd 'Start request repeated too quickly'	systemd rate-limit triggered after repeated failed starts	Ran systemctl reset-failed jenkins before restarting
Ansible inventory 'Unable to parse' as jenkins user	/home/ubuntu directory permissions (750) blocked traversal by jenkins user	chmod 755 /home/ubuntu
SSH 'Permission denied (publickey)' for jenkins user	jenkins SSH private key not readable by the ubuntu user (expected; jenkins owns the key)	Ran the Ansible/SSH test as 'sudo -u jenkins'
Jenkins git fetch error: couldn't find remote ref refs/heads/master	Branch specifier was */master but repo's default branch is main	Updated Branch Specifier to */main
ERROR: dockerhub-creds (credential not found)	Docker Hub credential not yet created in Jenkins	Added Username/Password credential with ID dockerhub-creds
Jenkinsfile git fetch 400 error	Placeholder URL <your-org>/banking-app.git left unedited in Jenkinsfile	Replaced placeholders with actual GitHub repo and Docker Hub usernames
Docker build: 'mvn: not found'	Multi-stage Dockerfile attempted to run Maven inside a JDK image without Maven installed	Switched to single-stage Dockerfile copying the JAR already built by Jenkins
Ansible: 'Unable to find any of pip3 to use'	pip3 not installed on app-server; playbook used the pip module	Installed python3-pip and python3-docker via the apt module
curl: Failed to connect to app-server:8989	Security group did not allow inbound traffic on port 8989	Added inbound rule for port 8989 (jenkins-server and public access)
curl returns empty / HTTP 302	curl -f does not follow redirects; /bank-api/swagger-ui.html redirects to /bank-api/swagger-ui/index.html	Updated verification step to use curl -fL against the index.html path
App listening on port 8080 instead of 8989	application.properties was not committed to the repository, so	Committed src/main/resources/application.prop

	Spring Boot used its default port	erties with server.port=8989
GitHub webhook did not trigger Jenkins build	Payload URL was malformed (duplicated http:// prefix and stray slash)	Corrected Payload URL to http://<jenkins-ip>:8080/github-webhook/ and re-tested

12. Conclusion

This project successfully demonstrates a complete, automated CI/CD pipeline for a Spring Boot banking microservice, integrating GitHub, Jenkins, Maven, Docker, Docker Hub, and Ansible to deploy to an AWS EC2 environment. Each push to the main branch automatically triggers the pipeline: the application is built and tested, packaged into a Docker image, pushed to a container registry, and deployed to the target server using Ansible, with an automated verification step confirming the application is reachable via its Swagger UI.

This workflow reduces manual deployment effort, shortens release cycles, and provides a repeatable, auditable process — directly addressing BankPro's goal of improving reliability, scalability, and service availability for its banking and payment platform.

12.1 Possible Future Enhancements

- Restrict security group rules to specific IP ranges instead of 0.0.0.0/0 for production use.
- Add a staging environment and promote builds through staging before production deployment.
- Use Docker Compose or Kubernetes for multi-container orchestration as the application grows.
- Add automated integration tests and code quality scanning (e.g., SonarQube) as pipeline stages.
- Use AWS Secrets Manager or Parameter Store for managing credentials instead of Jenkins credentials store.